17º Congresso de Inovação, Ciência e Tecnologia do IFSP – 2026
Campus Guarulhos

Sistemas Embarcados e IoT / RTOS: Como o First-Fit e o Best-Fit se comportam em microcontroladores com recursos extremamente limitados (como ESP32 rodando sistemas de tempo real (FreeRTOS))?

RESUMO: Este trabalho avalia e compara o desempenho das estratégias de alocação de memória dinâmica First-Fit e Best-Fit em sistemas embarcados de tempo real, utilizando o microcontrolador ESP32 e o sistema operacional FreeRTOS como ambiente experimental. A pesquisa se justifica pela escassez de estudos que avaliem essas estratégias diretamente em hardware embarcado real, considerando múltiplas métricas de desempenho de forma conjunta. Como procedimento, as implementações originais dos algoritmos foram extraídas do FreeRTOS-Kernel oficial e integradas como um componente isolado no ESP-IDF, com heap próprio de 64 KB. Um programa de carga de trabalho controlada, com 2.000 iterações de alocação e desalocação de blocos de tamanhos variados e liberação em ordem aleatória, foi executado 30 vezes em duas placas distintas, totalizando 58.068 observações, analisadas em Python. Os resultados mostraram desempenho antagônico entre as estratégias: o Best-Fit apresentou tempo médio, desvio-padrão e pior caso menores nas operações de alocação, enquanto o First-Fit foi superior nas operações de desalocação. Conclui-se que nenhuma estratégia é uniformemente superior, e que a escolha adequada depende do perfil de uso de memória predominante na aplicação, fornecendo um critério prático para desenvolvedores de sistemas embarcados de recursos limitados.

PALAVRAS-CHAVE: alocação de memória; fragmentação; desempenho; previsibilidade; heap; baixo custo.

Embedded Systems and IoT / RTOS: How Do First-Fit and Best-Fit Behave in Extremely Resource-Constrained Microcontrollers (such as ESP32 Running Real-Time Systems (FreeRTOS))?

ABSTRACT: This work evaluates and compares the performance of the First-Fit and Best-Fit dynamic memory allocation strategies in real-time embedded systems, using the ESP32 microcontroller and the FreeRTOS operating system as the experimental environment. The research is justified by the scarcity of studies that evaluate these strategies directly on real embedded hardware, jointly considering multiple performance metrics. As procedure, the original algorithm implementations were extracted from the official FreeRTOS-Kernel and integrated as an isolated component within ESP-IDF, with a dedicated 64 KB heap. A controlled workload program, with 2,000 iterations of allocation and deallocation of blocks of varying sizes and release in random order, was executed 30 times across two different boards, totaling 58,068 observations, analyzed in Python. Results showed antagonistic performance between the strategies: Best-Fit presented lower mean time, standard deviation, and worst case in allocation operations, while First-Fit was superior in deallocation operations. It is concluded that neither strategy is uniformly superior, and that the appropriate choice depends on the predominant memory usage profile of the application, providing a practical criterion for developers of resource-constrained embedded systems.

KEYWORDS: memory allocation; fragmentation; performance; predictability; heap; low cost.

INTRODUÇÃO

Os sistemas embarcados ocupam papel central na computação, sustentando a expansão da Internet das Coisas (IoT) sob restrições severas de memória e processamento (Musaddiq et al., 2018). Para lidar com essas limitações, consolidaram-se sistemas operacionais de tempo real (RTOS) enxutos, como o FreeRTOS, amplamente adotado em microcontroladores de baixo custo como o ESP32 (Goyette, 2007). Um dos aspectos mais sensíveis é o gerenciamento de memória dinâmica: as alocações devem possuir tempo de execução previsível, já que atrasos podem comprometer prazos críticos da aplicação (Puaut, 2002). Entre as estratégias clássicas, First-Fit e Best-Fit seguem como referência, mas seu comportamento em termos de fragmentação e tempo de resposta varia conforme a carga de trabalho, sem resultados consistentes na literatura (Shore, 1975). Apesar da relevância do tema, a maioria dos estudos avalia apenas o desempenho médio dos alocadores, negligenciando o pior caso, decisivo para sistemas de tempo real crítico (Puaut, 2002), lacuna agravada em microcontroladores sem unidade de gerenciamento de memória (MMU). Ramakrishna et al. (2008) destacam a escassez de avaliações experimentais que considerem simultaneamente tempo de resposta e uso de memória sob cargas realistas, enquanto estudos recentes no ESP32 tratam a alocação como variável secundária, sem aprofundar a comparação entre estratégias (Tanović; Mezei, 2024). Diante disso, este trabalho objetiva avaliar e comparar o desempenho das estratégias First-Fit e Best-Fit em ambiente embarcado de tempo real, utilizando ESP32 e FreeRTOS, considerando tempo de alocação e desalocação, previsibilidade (pior caso e percentis) e comportamento do heap sob cargas controladas.

MATERIAL E MÉTODOS

O experimento foi conduzido em duas placas ESP32 DevKitC (módulo ESP32-WROOM-32, dual-core, 4 MB de flash), utilizando o framework ESP-IDF v5.3.1, que já disponibiliza o FreeRTOS como núcleo do sistema. O uso de duas unidades físicas teve como objetivo controlar a variável de hardware, permitindo verificar se o comportamento observado se repete entre placas do mesmo modelo, e não é peculiaridade de uma unidade específica.

Investigação preliminar do ambiente (via menuconfig e inspeção do código-fonte) constatou que, a partir das versões 4.x/5.x, o ESP-IDF substituiu o gerenciador de heap clássico do FreeRTOS (arquivos heap_1.c a heap_5.c) por um alocador próprio da fabricante, baseado em TLSF (Two-Level Segregated Fit), não sendo mais possível alternar nativamente entre First-Fit e Best-Fit pela configuração padrão. Para viabilizar a comparação pretendida, os arquivos originais heap_4.c (First-Fit, com coalescência de blocos livres) e heap_2.c (Best-Fit, sem coalescência) foram obtidos diretamente do repositório oficial do FreeRTOS-Kernel e integrados ao projeto como um componente isolado, com heap próprio e independente do heap do sistema (64 KB fixos para cada estratégia), evitando qualquer interferência entre os alocadores testados e o restante do sistema operacional.

Para a geração de carga de trabalho, foi implementado um programa em C que executa 2.000 iterações de operações de alocação e desalocação por execução, com tamanhos de bloco distribuídos de forma ponderada (70% pequenos, até 144 bytes; 25% médios, até 768 bytes; 5% grandes, até 2.048 bytes) e liberação em ordem aleatória (40% de probabilidade por iteração), de modo a induzir fragmentação de forma controlada, conforme discutido por Shore (1975) e Puaut (2002). Cada operação teve seu tempo de execução medido em microssegundos, via esp_timer_get_time(), e o heap livre foi registrado após cada operação. Os dados foram exportados em formato CSV pela saída serial e capturados diretamente em arquivo no computador host, evitando gravação em memória flash durante o teste, o que distorceria as medições de tempo.

Cada estratégia (First-Fit e Best-Fit) foi executada 10 vezes na primeira placa e 5 vezes na segunda, totalizando 30 execuções e 58.068 linhas de dados válidas após o processamento. A análise dos dados foi realizada em Python, com a biblioteca pandas, calculando-se média, desvio padrão, pior

caso (valor máximo) e percentil 99 do tempo de alocação e desalocação para cada estratégia, além da geração de gráficos comparativos com a biblioteca matplotlib.

RESULTADOS E DISCUSSÃO

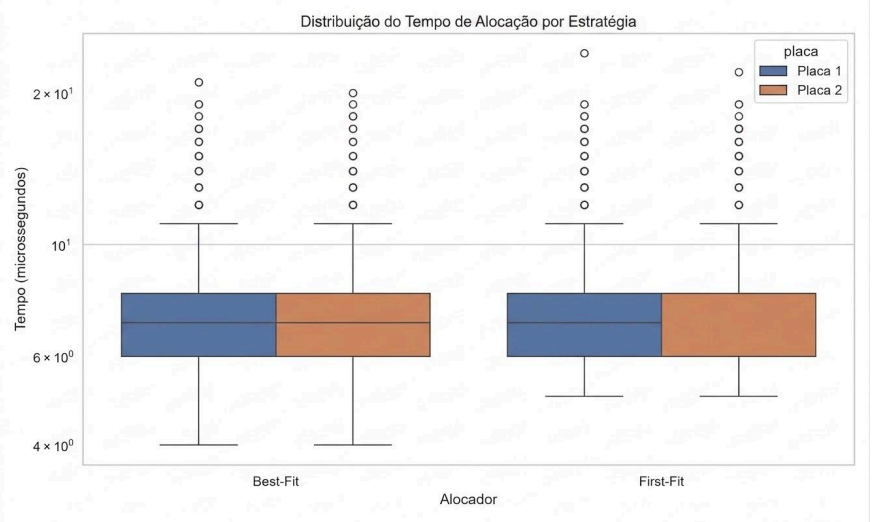
A Tabela 1 apresenta as estatísticas de tempo de execução (em microssegundos) das operações de alocação e desalocação, consolidadas a partir das 58.068 observações coletadas nas duas placas.

TABELA 1. Estatísticas de tempo de alocação e desalocação (µs) por estratégia.

Operação	Estratégia	Média	Desvio-padrão	Pior caso	Percentil 99
Alocação	Best-Fit	6,95	1,81	21	13
Alocação	First-Fit	7,35	2,32	24	15
Desalocação	Best-Fit	6,22	1,31	20	10
Desalocação	First-Fit	5,63	0,72	16	8

Os resultados mostram um padrão que não favorece uniformemente nenhuma das duas estratégias, corroborando a observação de Shore (1975) de que a comparação entre First-Fit e Best-Fit não produz resultados consistentes entre si, variando conforme o tipo de operação e as características da carga de trabalho. Para as operações de alocação, o Best-Fit apresentou desempenho superior em todos os indicadores: menor tempo médio (6,95 µs contra 7,35 µs), menor variabilidade (desvio-padrão de 1,81 µs contra 2,32 µs), melhor pior caso (21 µs contra 24 µs) e melhor percentil 99 (13,0 µs contra 15,0 µs). Esse resultado é relevante à luz da crítica de Puaut (2002) de que grande parte dos estudos negligencia o pior caso em favor do desempenho médio: aqui, o Best-Fit se mostrou não apenas mais rápido em média, mas também mais previsível, característica desejável em sistemas de tempo real.

FIGURA 1. Distribuição do tempo de alocação (µs) por estratégia, segmentada por placa.



A Figura 1 detalha essa diferença, apresentando a distribuição do tempo de alocação segmentada por placa. As caixas (intervalo interquartil) do Best-Fit posicionam-se discretamente abaixo das do First-Fit em ambas as unidades testadas, e a cauda de valores atípicos (outliers) do First-Fit se estende a valores mais altos, incluindo o pior caso absoluto de todo o experimento (24 µs, registrado na Placa 1). É importante destacar que, para cada estratégia, as distribuições da Placa 1 e da Placa 2 praticamente se sobrepõem, tanto a mediana quanto a amplitude das caixas são muito semelhantes entre as duas unidades. Essa consistência reforça que a diferença de desempenho observada decorre da estratégia de alocação em si, e não de variações de fabricação entre unidades

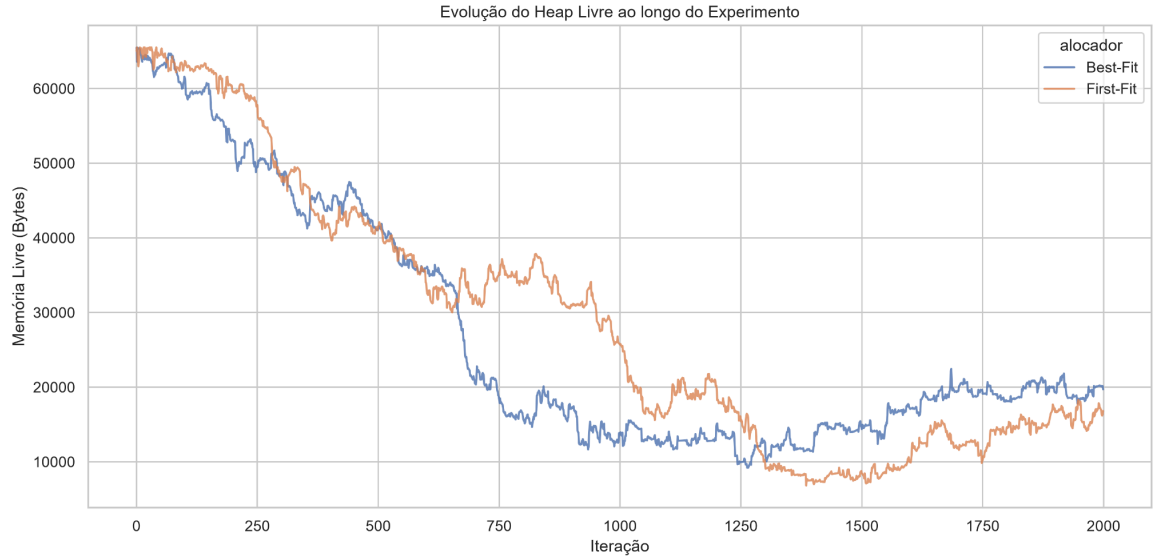
de hardware do mesmo modelo, ponto de atenção já discutido na literatura sobre microcontroladores de baixo custo (Deligiannis; Kornaros, 2016).

Já nas operações de desalocação, o cenário se inverte: o First-Fit apresentou melhor desempenho em todos os indicadores (média de 5,63 µs contra 6,22 µs; pior caso de 16 µs contra 20 µs; percentil 99 de 8,0 µs contra 10,0 µs). Uma hipótese para essa inversão é a diferença estrutural entre as duas implementações: o heap_4.c (First-Fit) realiza coalescência de blocos adjacentes livres no momento da liberação, o que, para o padrão de fragmentação induzido no experimento, aparentemente resultou em um processo de liberação mais eficiente do que a reinserção ordenada por tamanho utilizada pelo heap_2.c (Best-Fit), que não realiza coalescência.

Esse comportamento antagônico entre as operações reforça o argumento de Ramakrishna et al. (2008) sobre a necessidade de avaliações experimentais que considerem simultaneamente múltiplas métricas de desempenho, e não apenas uma métrica isolada: uma análise restrita apenas ao tempo de alocação levaria à conclusão de que o Best-Fit é preferível, enquanto uma análise restrita à desalocação indicaria o oposto.

A Figura 2 ilustra a evolução da memória heap livre ao longo das iterações.

FIGURA 2. Evolução da memória heap livre ao longo das iterações, por estratégia de alocação.



Essa evolução, ao longo das 2.000 iterações de uma execução representativa de cada estratégia, permite interpretar qualitativamente a dinâmica de fragmentação por trás dos números da Tabela 1. Ambas as curvas partem de aproximadamente 65.000 bytes livres e decaem de forma semelhante até por volta da iteração 600. A partir desse ponto, o heap do Best-Fit despenca de forma mais abrupta, atingindo seu ponto mínimo (cerca de 9.000–10.000 bytes) próximo à iteração 1.250; o First-Fit tem sua queda mais acentuada retardada em aproximadamente 100 iterações, porém, quando ocorre, ultrapassa o Best-Fit e atinge o menor valor de heap livre observado em todo o experimento (cerca de 7.000 bytes), próximo à iteração 1.350. Na segunda metade do teste, as duas curvas se recuperam de forma oscilante, resultado esperado, já que a probabilidade de liberação (40% por iteração) tende a reequilibrar o heap ao longo de uma janela suficientemente longa, mas o Best-Fit estabiliza-se em um patamar de memória livre discretamente superior (aproximadamente 18.000–20.000 bytes) ao do First-Fit (aproximadamente 15.000–17.000 bytes) nas iterações finais.

Esse comportamento sugere que, apesar de realizar coalescência de blocos adjacentes, o First-Fit não necessariamente produz menor fragmentação ao longo do tempo: a coalescência une apenas blocos fisicamente adjacentes, sem eliminar espaços intercalados por blocos ainda ativos, fenômeno cuja correção por compactação de memória é inviável em microcontroladores sem unidade de gerenciamento de memória (MMU), como o ESP32 (DELIGIANNIS; KORNAROS, 2016). A maior amplitude de variação do First-Fit entre pico e vale do heap livre, evidenciada na Figura 2, é consistente com a maior variabilidade de tempo (desvio-padrão) observada para essa estratégia na Tabela 1, sugerindo um comportamento de uso de memória menos previsível ao longo da execução.

Em conjunto, os resultados indicam que nenhuma das duas estratégias é uniformemente superior: o Best-Fit demonstrou melhor desempenho e maior previsibilidade na alocação, além de manter um patamar de memória livre discretamente mais estável ao final do experimento; o First-Fit, por sua vez, foi mais eficiente na desalocação. A consistência dos resultados entre as duas placas testadas (Figura 1) fortalece a validade interna do experimento, permitindo atribuir as diferenças observadas às estratégias de alocação avaliadas, e não a peculiaridades de hardware de uma unidade específica.

CONCLUSÕES

Os resultados obtidos permitem afirmar que o objetivo geral do trabalho, avaliar e comparar o desempenho das estratégias First-Fit e Best-Fit em um ambiente embarcado de tempo real baseado em ESP32 e FreeRTOS, foi atingido, ainda que a resposta encontrada seja mais nuançada do que a simples indicação de uma estratégia superior à outra. Ao contrário de uma expectativa inicial de que um dos algoritmos se sobressairia de forma consistente, o experimento revelou um comportamento antagônico entre as operações: o Best-Fit mostrou-se mais rápido e previsível nas alocações, enquanto o First-Fit foi mais eficiente nas desalocações, evidenciando que a escolha da estratégia mais adequada depende do perfil de uso de memória predominante na aplicação de tempo real em questão.

Esse achado confronta diretamente a lacuna apontada na introdução: a literatura pouco explora avaliações experimentais conduzidas em hardware embarcado real que considerem múltiplas métricas de desempenho de forma conjunta. Ao medir não apenas o tempo médio, mas também o pior caso e o percentil 99 para ambas as operações, o trabalho evidenciou que a previsibilidade, aspecto central para sistemas de tempo real crítico, também se comporta de maneira distinta conforme a operação analisada, reforçando que decisões de projeto baseadas apenas no tempo médio de alocação podem levar a conclusões equivocadas quanto ao comportamento real do sistema. A consistência dos resultados entre as duas placas testadas indica ainda que as diferenças observadas são atribuíveis às estratégias de alocação avaliadas, e não a variações de fabricação do hardware utilizado.

Conclui-se, portanto, que para aplicações embarcadas de tempo real dominadas por operações de alocação, com liberação de memória menos frequente, o Best-Fit tende a oferecer maior previsibilidade; já para aplicações com padrão intenso de liberação de blocos, o First-Fit tende a ser mais adequado. Essa conclusão fornece um critério prático, ausente na literatura consultada, para orientar desenvolvedores de sistemas embarcados de recursos limitados na escolha da estratégia de alocação de memória dinâmica mais adequada ao perfil de sua aplicação.

AGRADECIMENTOS

Agradeço aos Institutos Federais pela oportunidade de formação e pelo incentivo à pesquisa e à iniciação científica, em especial ao Campus Lages, pelo amparo e apoio ao longo do desenvolvimento deste trabalho. Agradeço também ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), pela estrutura oferecida e pela oportunidade de apresentação desta pesquisa no CONICT 2026.

REFERÊNCIAS

DELIGIANNIS, I.; KORNAROS, G. Adaptive memory management scheme for MMU-less embedded systems. In: IEEE SYMPOSIUM ON INDUSTRIAL EMBEDDED SYSTEMS (SIES), 11., 2016, Krakow. Proceedings [...]. Krakow: IEEE, 2016. p. 1-8. DOI: 10.1109/SIES.2016.7509439. Disponível em: https://ieeexplore.ieee.org/abstract/document/7509439. Acesso em: 29 jul. 2026.

GOYETTE, R. An analysis and description of the inner workings of the FreeRTOS kernel. Ottawa: Carleton University, Department of Systems and Computer Engineering, 2007. Relatório técnico (SYSC5701: Operating System Methods for Real-Time Applications). Disponível em: https://benhup.com/download/eduntech/stk500_stk501/FreeRTOSPaper.pdf. Acesso em: 28 jul. 2026.

MUSADDIQ, A.; ZIKRIA, Y. B.; HAHM, O.; YU, H.; BASHIR, A. K.; KIM, S. W. A survey on resource management in IoT operating systems. IEEE Access, v. 6, p. 8459-8482, 2018. DOI: 10.1109/ACCESS.2018.2808324. Disponível em: https://ieeexplore.ieee.org/abstract/document/8300305. Acesso em: 29 jul. 2026.

PUAUT, I. Real-time performance of dynamic memory allocation algorithms. In: EUROMICRO CONFERENCE ON REAL-TIME SYSTEMS, 14., 2002, Vienna. Proceedings [...]. Vienna: IEEE, 2002. p. 41-49. DOI: 10.1109/EMRTS.2002.1019184. Disponível em: https://ieeexplore.ieee.org/abstract/document/1019184. Acesso em: 29 jul. 2026.

RAMAKRISHNA, M.; KIM, J.; LEE, W.; CHUNG, Y. Smart dynamic memory allocator for embedded systems. In: INTERNATIONAL SYMPOSIUM ON COMPUTER AND INFORMATION SCIENCES, 23., 2008, Istanbul. Proceedings [...]. Istanbul: IEEE, 2008. p. 1-6. DOI: 10.1109/ISCIS.2008.4717922. Disponível em: https://ieeexplore.ieee.org/abstract/document/4717922. Acesso em: 29 jul. 2026.

SHORE, J. E. On the external storage fragmentation produced by first-fit and best-fit allocation strategies. Communications of the ACM, v. 18, n. 8, p. 433-440, ago. 1975. DOI: 10.1145/360933.360949. Disponível em: https://dl.acm.org/doi/abs/10.1145/360933.360949. Acesso em: 29 jul. 2026.

TANOVIĆ, A.; MEZEI, I. Embedded parallel K-means algorithm evaluation on ESP32 across various memory allocations. In: IEEE EAST-WEST DESIGN & TEST SYMPOSIUM (EWDTS), 2024, Yerevan. Proceedings [...]. Yerevan: IEEE, 2024. p. 1-7. DOI: 10.1109/EWDTS63723.2024.10873639. Disponível em: https://ieeexplore.ieee.org/abstract/document/10873639. Acesso em: 29 jul. 2026.